Permuting the six AX heads: command completeness and arithmetic transfer

Technical lemma for a proposed layout change

13 November 2026

This note is a mathematical permission to change one particular output order, not a certificate for an untested transition table. Its hypotheses must be established by the source macro and physical runtime checks for the selected candidate. It does not claim a new kernel-checked theorem.

1 The precise allowed change

Write $\langle a, b\rangle$ for the source Cantor pair, $I(a,b) = 4\langle a,b\rangle$, $M(a,b) = 4\langle a,b\rangle + 1$, $E(a,b) = 4\langle a,b\rangle + 3$, and $U(d,h) = 4\langle h,d\rangle + 2$. Let $q_i(D, A, H)$, $1 \le i \le 6$, be the six published PA196 heads, with the unused A argument ignored for heads 1 and 5. For a fixed permutation π of $\{1,\dots,6\}$, define the modified AX command on the store (t, w, d, h) by

$$\mathsf{AX}_\pi(c,x,y)(t,w,d,h) = \big(G(H,D,c), [q_{\pi(1)},\dots,q_{\pi(6)},\rho,U(d,h),t]_w, 0, 0\big),$$

where $D = d+x$, $H = h+y$, each q_i has arguments (D, c, H), $\rho = \rho_8(D,c,H,d)$, G is the unchanged guarded gate, and $[a_1,\dots,a_k]_w = \langle a_1, \langle a_2, \dots, \langle a_k, w\rangle \dots\rangle\rangle$. Thus π denotes *top-to-bottom* stack order; the published order is $(6,5,4,3,2,1)$. The gate remains the current top, the propositional and generalization heads remain in positions 7 and 8, the incoming top remains in position 9, and both caches are cleared. Shift and MP, their parameter conventions, the source scheduler, and the starting store are unchanged.

2 Exact selectors without accidental halts

Shift halts precisely when $t+h = 1$; otherwise it maps (t,w,d,h) to $(h, \mathrm{right}(w), d, \mathrm{left}(w))$. Each q_i, including the existential head q_5, has an outer implication constructor and is a multiple of 4. The same is true of ρ and of G, including the guarded value $G = 0$. Consequently none is 1, and no sum of two such values is 1. The fixed generalization head $U(d,h)$ is at least 2. These elementary identities hold in every model of PA.

Lemma 1 (Order-independent stack macros)**.** *For every π and every incoming store, including $t = 1$, the following exact executions are live and have the displayed endpoints:*

$$\mathsf{AX}_\pi(c,x,y)\,\mathsf{Shift}^8(t,w,d,h) = (\rho, \langle t, w\rangle, 0, U(d,h)),$$
$$\mathsf{AX}_\pi(c,x,y)\,\mathsf{Shift}^9(t,w,d,h) = (U(d,h), w, 0, t).$$

Here a word is written in execution order. Hence the unchanged words

$$\mathsf{pop} = \mathsf{AX}_\pi(0,0,0)\,\mathsf{Shift}^{10},$$
$$\mathsf{push} = \mathsf{AX}_\pi(0,0,0)\,\mathsf{Shift}^8\mathsf{MP}(0),$$
$$\mathsf{discard} = \mathsf{MP}(1)\mathsf{AX}_\pi(1,0,0)\,\mathsf{Shift}^9\mathsf{MP}(0)$$

satisfy, respectively,

$$\mathsf{pop}(t,\langle a,w\rangle,d,h) = (t,w,0,a),$$
$$\mathsf{push}(t,w,d,h) = (0,\langle t,w\rangle,0,0),$$
$$\mathsf{discard}(t,w,0,0) = (0,w,0,0).$$

These statements require no consistency hypothesis.

Proof. During the first eight Shifts, both tested entries are drawn from zero, the gate, and the first eight generated heads, so the halt test cannot succeed. The ninth test includes $U(d,h) \geq 2$. The tenth test, used for pop, still includes this universal head, so it is safe even when the saved incoming top is 1. Only after that test is the incoming top restored. This is why pop is total even though a bare Shift on arbitrary formula entries need not be total. Cantor decoding now gives the two displayed identities and the pop endpoint.

For push, the last MP compares an implication code, divisible by 4, with $U(d,h) \equiv 2 \pmod 4$; the test fails and leaves the saved top on the stack. In discard, its first MP fails because $I(t,1) \neq 0$, leaving $(0,w,1,0)$. The nine-Shift identity gives $(U(1,0),w,0,0)$; the last MP fails because $I(U(1,0),0) \neq 0$. All arguments are finite algebraic derivations uniform in the incoming parameters, not tests on bounded numerical samples. □

For $1 \leq j \leq 6$ define

$$\mathrm{select}_\pi(j,D,H,A) = \mathsf{discard}\,\mathsf{AX}_\pi(A,D,H)\,\mathsf{Shift}^{j+1}\mathsf{pop}^{8-j}.$$

The exact endpoint from $(t,w,0,0)$ is

$$\big(q_{\pi(j)}(D,A,H),w,0,0\big).$$

Indeed, the initial discard makes the saved top zero; $j+1$ Shifts expose the desired head; the remaining pops preserve it and consume the suffix through that saved zero. Thus head i is selected at the fixed position $j = \pi^{-1}(i)$. The selector has $102 - 10j \leq 92$ commands. The existing selectors for ρ_8 and the guarded gate are unchanged, as are generalization and the MP suffix. They use only the two order-independent Shift identities and the total macros above. The possible early halt in the MP suffix remains the intentional case in which its already derived antecedent is code 1.

3 A uniform derivation-to-halting-word map

Lemma 2 (Modified compiler)**.** *Let $\mathcal{C}$ be the unchanged native proof calculus. There is a primitive recursive compiler $p \mapsto C_\pi(p)$ such that PA proves, for every internally verified $\mathcal{C}$ derivation p of formula code a and every t,w, that execution from $(t,w,0,0)$ either halts or returns $(a,w,0,0)$. Therefore, if p is a derivation of 1, the word*

$$W_\pi(p) = \mathsf{AX}_\pi(0,0,0)\,C_\pi(p)\,\mathsf{Shift}$$

halts from every *incoming live store.*

Proof. Use the published proof compiler, replacing its six fixed head positions by $\pi^{-1}(i)$. The gate, restricted propositional, generalization and MP cases keep their command words. At a leaf, the new selector lemma proves the required endpoint. At an MP node, compile the major proof, push its result, compile the minor proof, push its result, and run the unchanged MP suffix. At a generalization node run the unchanged generalization word after compiling its premise. If a premise execution has already halted, the absorbing command semantics preserves that result. Induction on the coded derivation proves the assertion.

The initial AX in W_π clears both caches for arbitrary incoming d,h, so the compiler theorem applies with arbitrary new top and stack. If it returns, its top is 1 and its h cache is zero, and the final Shift halts. This handles dirty incoming caches and is strong enough for the scheduler, which does not reset the store between its candidate words.

The only new constants in the proof generator are six numbers between 1 and 6. Its recursion and word encoding are the original primitive recursive operations. For an expanded derivation tree with n nodes, the command word has length at most $112n + 2$; large formula parameters affect command codes, not that word-length bound. PA verifies the recursion by induction on the coded proof, including internally finite proofs of nonstandard length. No appeal to truth of all internally provable formulas is involved. □

The original source word encoder still applies: if E folds a command word into its source Cantor tail encoding, the candidate index is $E(W_\pi(p)) - 1$. The reset prefix puts this index in the enumerated range. The exact decoded-word theorem and candidate enumeration induction therefore give a source halt from any native refutation. These are uniform arithmetical maps, with the same argument for nonstandard words in a PA model; one must not replace this with an external assertion that every standard word is eventually tried.

4 Soundness and transfer to a physical machine

An order-independent proof annotation also goes in the other direction. Starting from the zero store, attach a PA proof to the current top, the h cache and each finite stack entry; zero tails carry a fixed proof of code-zero truth. The PA interpretation is the direct native-code interpretation from the accompanying soundness note. At AX, the six head proofs are merely reordered along with their formula codes. The gate proof, restricted propositional proof, proof of $U(d,h)$ obtained by generalization, and saved-top proof are unchanged. MP combines proofs on success and installs the fixed truth proof on failure. A nonhalting Shift only moves existing proof witnesses. If a Shift halts, $t+h=1$ implies $t=1$ or $h=1$, so one of the attached proofs is a proof of code 1. Its direct interpretation contradicts PA's theorem that a number is not its own binary-bit member. This constructs a PA refutation; it does not assume that PA proves its consistency.

Theorem 3 (Permutation transfer, conditional on implementation)**.** *Suppose a proposed binary machine M_π has a verified blank startup and PA-formalizable uniform positive instruction and command-macro contracts implementing exactly AX_π, the unchanged MP and Shift commands, and the unchanged complete scheduler. The AX contracts must return to the live command boundary for every natural parameter and may not expose an additional halt. Then, with the fixed PA-to-native proof translation and the direct native-to-PA syntactic translation,*

$$\mathrm{PA} \vdash H_{M_\pi} \leftrightarrow \neg\,\mathrm{Con(PA)}, \qquad \mathrm{PA} \vdash \neg H_{M_\pi} \leftrightarrow \mathrm{Con(PA)}.$$

In particular it has the same arithmetical halting statement as the baseline, given the accompanying paper-level reverse-soundness proof for the baseline. These conclusions also hold in the standard model.

Proof. From a PA refutation, use the unchanged PA-to-native translation, the modified compiler W_π, the exact source word encoder, and candidate enumeration. Positive source and physical contracts produce a literal halting trace.

Conversely, under the internal hypothesis Con(PA), the proof annotation makes every command successor live. The scheduler's positive fresh-candidate and positive-residual contracts then build arbitrarily long live source traces, and the physical contracts build arbitrarily long live literal traces. If a literal halt existed at time T, building $T+1$ positive segments would contradict determinism at that halt. If a coded source halt is represented by an absorbing self-loop, its absorbed endpoint is likewise incompatible with a later live endpoint. This is an induction inside PA; it makes no assumption of external standardness for T or for formula codes. Thus PA proves both directions claimed. □

Implementation boundary. An instruction-level check against a changed source proves only that this source is executed correctly. In addition one must prove the changed AX macro has exactly the output displayed in Section 1, with the same argument values, live return, gate guard, and fixed tail. Source order is now different, so an assertion of identical ordinal control flow cannot replace that macro check. A successful check of every proposed permutation is useful for search; only the chosen candidate's complete contracts and startup justify a final machine claim.

5 Optional exchange of the first two antecedents

The same argument permits, independently for heads 3, 4 and 6, replacing $I(\alpha, I(\beta,\gamma))$ by $I(\beta, I(\alpha,\gamma))$. Here the triples are

i	α	β	γ
3	$M(H,D)$	$E(H,A)$	$M(A,D)$
4	$E(H,D)$	$M(A,H)$	$M(A,D)$
6	$U(D,A)$	$U(D,I(A,H))$	$U(D,H)$.

This is not a claim that either membership head may be deleted. The exchanged head is a different formula, so the native proof compiler needs one more explicit translation.

The restricted propositional head alone, with MP, uniformly derives

$$I\big(I(\alpha,I(\beta,\gamma)),I(\beta,I(\alpha,\gamma))\big).$$

The existing finite proof template is `PA196.Logic.Dominated.exchange` in `PA196/Logic/ImplicationBasis.lean`; its underlying rules use only the restricted ρ_8 axioms and MP. Interchanging α, β gives the reverse implication. Thus replacing any of these three axiom schemas by its exchanged version admits primitive recursive proof translations in both directions.

For completeness, translate an old head leaf to the selected new head followed by the appropriate exchange proof and MP. The exchange proof itself uses only the unchanged propositional selector, so this construction is not circular. For soundness, derive a newly emitted exchanged head from its old head proof using the forward exchange template. Each new head is still divisible by 4. Every stack-macro and no-accidental-halt calculation above therefore remains valid. The same all-model induction proves the transfer theorem for all $6!\,2^3 = 5{,}760$ combinations, conditional on their exact source and physical contracts. The bound $112n + 2$ above applies to pure permutations; expanding the fixed exchange templates gives a larger fixed linear bound for these additional variants. No numeric or semantic sampling is needed to justify these logical transformations.

Existing formal templates. The fixed baseline is commit `407cdc356f6b586635e71ac92be394255f4bfea4` of jamesyc/BusyBeaver. The relevant declarations are `Commands.Words.ax_shift8`, `ax_shift9`, `pop_total`, `push_total`, `discard_total`, the six theorems of `Commands.HeadSelectors`, `Commands.Compiler.compile_result` and `haltWord_universal`. Their all-model arithmetic templates are in `Arithmetic/WordAlgebra.lean`, `WordSelectors.lean`, `CompilerLeaves.lean`, and `CompilerHaltingWord.lean`. The exact word encoding and enumeration are developed in `Arithmetic/Scheduler/ActualWordEncodingBlueprint.lean`, `SourceCommandWordTraversal.lean`, `ActualCandidateBlueprint.lean`, and `InternalCodedSourceCandidate.lean`. The permutation modifications described here have not been implemented and checked as new Lean declarations.